

Guía completa: Máquina virtual Ubuntu para práctica DevOps CI/CD

Procedimiento exclusivo para VirtualBox + Ubuntu. Esta guía parte de que en tu PC ya realizaste los pasos 1 al 9 de la práctica: Git, Docker, proyecto, GitHub y GitHub Actions básico. Aquí se prepara la máquina virtual para funcionar como servidor y posteriormente recibir el despliegue automático.

Objetivo

Al terminar, la máquina virtual tendrá Ubuntu, Git, Docker, Docker Compose y SSH. También tendrá el proyecto clonado desde GitHub y podrá ejecutar la aplicación mediante Docker. Finalmente se dejará preparada para que GitHub Actions pueda conectarse por SSH y ejecutar el despliegue.

Arquitectura final

```
PC / GitHub
  |
  | GitHub Actions + SSH
  v
VirtualBox -> Ubuntu
  |
  |-- Git
  |-- Docker
  |-- Docker Compose
  |-- OpenSSH
  |-- practica-devops-cicd
    |
    |-- Nginx + index.html
    |-- puerto 8080
```

PARTE 1 — Crear o preparar la máquina virtual

1. Abre VirtualBox y selecciona tu máquina virtual Ubuntu. Iníciala e inicia sesión.
2. Recomendación para la red: para las primeras pruebas usa un adaptador que permita que tu PC y la VM se comuniquen. Si usas Adaptador puente, la VM normalmente recibe una IP de tu misma red. Si usas NAT, puede ser necesario configurar reglas de reenvío de puertos para acceder desde el PC.
3. Abre una terminal dentro de Ubuntu con Ctrl + Alt + T.

PARTE 2 — Actualizar Ubuntu

```
sudo apt update
sudo apt upgrade -y
```

Comprueba la versión:

```
lsb_release -a
```

PARTE 3 — Instalar Git

```
sudo apt install git -y
git --version
```

Configura el nombre y correo que usarás para Git:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@gmail.com"
git config --list
```

PARTE 4 — Instalar Docker y Docker Compose

Ejecuta los siguientes comandos en orden:

```
sudo apt update
sudo apt install ca-certificates curl -y
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
```

Comprueba la instalación:

```
docker --version
docker compose version
sudo systemctl status docker
```

Si Docker no está activo:

```
sudo systemctl start docker
sudo systemctl enable docker
```

Permitir Docker sin sudo

```
sudo usermod -aG docker $USER
sudo reboot
```

Después de reiniciar, prueba:

```
docker ps
docker run hello-world
```

Si aparece 'Hello from Docker!', Docker funciona correctamente.

PARTE 5 — Instalar y configurar SSH

SSH permitirá que tu PC se conecte a la VM y será el mecanismo que posteriormente utilizará GitHub Actions para el Deploy.

```
sudo apt install openssh-server -y
sudo systemctl status ssh
sudo systemctl enable ssh
```

Si SSH no está activo:

```
sudo systemctl start ssh
```

Obtener la IP de la VM

```
hostname -I
ip addr
```

Anota la IP, por ejemplo 192.168.1.100. No la sustituyas por este ejemplo: usa la IP real que muestre tu VM.

PARTE 6 — Probar conexión SSH desde tu PC

En tu PC, no dentro de la VM, abre una terminal y ejecuta:

```
ssh USUARIO_VM@IP_DE_LA_VM
```

Ejemplo:

```
ssh tadeo@192.168.1.100
```

Si aparece una pregunta de autenticidad, escribe yes y después la contraseña del usuario de Ubuntu.

Si consigues entrar a Ubuntu desde la terminal de tu PC, la comunicación PC → VM está funcionando.

PARTE 7 — Clonar el proyecto desde GitHub

Regresa a la terminal de la VM. Crea una carpeta para el proyecto:

```
mkdir -p ~/proyectos  
cd ~/proyectos
```

Clona el repositorio que ya subiste desde tu PC:

```
git clone https://github.com/TU_USUARIO/practica-devops-cicd.git
```

Entra al proyecto:

```
cd practica-devops-cicd  
ls
```

Debes encontrar archivos como Dockerfile, docker-compose.yml, index.html, README.md y la carpeta .github.

PARTE 8 — Probar Docker en la VM

Antes del despliegue automático, verifica que el servidor pueda ejecutar manualmente el proyecto.

```
cd ~/proyectos/practica-devops-cicd  
docker compose up -d --build  
docker ps
```

Prueba desde la propia VM:

```
curl http://localhost:8080  
curl -I http://localhost:8080
```

La segunda prueba debe devolver un estado HTTP como 200 OK.

Probar desde el navegador de tu PC

En el navegador de tu PC escribe:

```
http://IP_DE_LA_VM:8080
```

Ejemplo:

```
http://192.168.1.100:8080
```

Si aparece la aplicación DevOps, la VM ya funciona como servidor web.

PARTE 9 — Preparar acceso por clave SSH para GitHub Actions

Esta parte se realiza principalmente en tu PC. La clave privada se guardará como secreto de GitHub; nunca la publiques ni la subas al repositorio.

En tu PC genera una clave Ed25519:

```
ssh-keygen -t ed25519 -C "github-actions"
```

Puedes aceptar la ubicación predeterminada y, para esta práctica, dejar la contraseña de la clave vacía presionando Enter cuando se solicite.

Se crearán normalmente:

```
~/ssh/id_ed25519
~/ssh/id_ed25519.pub
```

La privada es id_ed25519. La pública es id_ed25519.pub.

Copiar la clave pública a la VM

Desde tu PC:

```
ssh-copy-id USUARIO_VM@IP_DE_LA_VM
```

Después prueba:

```
ssh USUARIO_VM@IP_DE_LA_VM
```

Debe permitirte entrar sin pedir la contraseña de Ubuntu. Si no funciona, no continúes con GitHub Actions todavía: primero corrige SSH.

PARTE 10 — Configurar Secrets en GitHub

En GitHub abre tu repositorio y entra a: Settings → Secrets and variables → Actions → New repository secret.

Crea estos tres secretos:

Secreto	Valor
SERVER_HOST	IP real de la VM
SERVER_USER	Usuario de Ubuntu
SERVER_SSH_KEY	Contenido completo de ~/.ssh/id_ed25519

Importante: SERVER_SSH_KEY debe contener la clave privada id_ed25519, no el archivo .pub. No compartas esta clave fuera de GitHub Secrets.

PARTE 11 — Preparar el comando de despliegue en la VM

El directorio del proyecto debe quedar en una ubicación estable. En esta guía usamos:

```
~/proyectos/practica-devops-cicd
```

Verifica:

```
cd ~/proyectos/practica-devops-cicd
pwd
```

El Deploy utilizará estos comandos:

```
git pull origin main
docker compose down
docker compose up -d --build
```

La práctica utiliza este mismo principio: actualizar el código, detener el servicio y reconstruir/levantar el contenedor.

PARTE 12 — Agregar Deploy al workflow

En tu PC edita el archivo:

```
.github/workflows/ci-cd.yml
```

Después de los jobs test y build, agrega:

```
deploy:
  name: Desplegar aplicación
  needs: build
  runs-on: ubuntu-latest
```

```

steps:
  - name: Conectar y desplegar
    uses: appleboy/ssh-action@v1.2.0
    with:
      host: ${ secrets.SERVER_HOST }
      username: ${ secrets.SERVER_USER }
      key: ${ secrets.SERVER_SSH_KEY }
      script: |
        cd ~/proyectos/practica-devops-cicd
        git pull origin main
        docker compose down
        docker compose up -d --build

```

Guarda el archivo y súbelo:

```

git add .
git commit -m "Agregar despliegue automático"
git push

```

PARTE 13 — Comprobar el pipeline

En GitHub abre Actions y busca el workflow CI-CD DevOps. El flujo esperado es:

```

PUSH
↓
TEST ✓
↓
BUILD ✓
↓
DEPLOY ✓
↓
SSH
↓
Ubuntu VM
↓
git pull
↓
Docker Compose
↓
Aplicación actualizada

```

Si TEST falla, BUILD no debe ejecutarse. Si BUILD termina correctamente, Deploy puede comenzar.

PARTE 14 — Hacer una prueba real

En tu PC cambia un texto de index.html. Por ejemplo, cambia el mensaje de la aplicación. Después ejecuta:

```

git add .
git commit -m "Actualizar aplicación"
git push

```

Ve a GitHub → Actions y observa TEST, BUILD y DEPLOY.

Cuando Deploy termine, abre desde tu PC:

```
http://IP_DE_LA_VM:8080
```

El cambio debe aparecer después de que Docker haya sido reconstruido.

PARTE 15 — Comandos de diagnóstico

Si la aplicación no aparece:

```

docker ps
docker compose logs
docker compose ps
curl -I http://localhost:8080

```

Si necesitas reconstruir manualmente:

```
cd ~/proyectos/practica-devops-cicd
docker compose down
docker compose up -d --build
```

Si necesitas comprobar Git:

```
git status
git log -1
git remote -v
```

Si necesitas comprobar SSH:

```
sudo systemctl status ssh
ss -tlnp | grep :22
```

PARTE 16 — Problema frecuente: GitHub no puede llegar a la VM

La prueba PC → VM no garantiza que GitHub Actions → VM funcione. GitHub Actions se ejecuta fuera de tu red local. Si la VM tiene una IP privada como 192.168.x.x, normalmente no es accesible directamente desde Internet.

Si el Deploy falla por conexión SSH, primero comprueba que PC → VM funciona. Después será necesario revisar la red de VirtualBox y, si corresponde, la configuración del router/port forwarding o utilizar una alternativa de red segura. No expongas SSH a Internet sin entender la configuración.

Checklist final

Elemento	Debe estar
Ubuntu en VirtualBox	✓
Git instalado	✓ git --version
Docker instalado	✓ docker --version
Docker Compose instalado	✓ docker compose version
Docker funcionando	✓ docker run hello-world
SSH funcionando	✓ systemctl status ssh
IP de la VM identificada	✓ hostname -I
PC → VM por SSH	✓ ssh usuario@IP
Proyecto clonado	✓ ~/proyectos/practica-devops-cicd
Aplicación en Docker	✓ localhost:8080
GitHub Secrets	✓ SERVER_HOST / USER / SSH_KEY
GitHub Actions	✓ TEST → BUILD → DEPLOY

Guion corto para explicar al maestro

“Preparé una máquina virtual con Ubuntu para utilizarla como servidor. Instalé Git, Docker, Docker Compose y SSH. Cloné desde GitHub el proyecto que desarrollé en mi PC y comprobé que podía ejecutarlo con Docker. Después configuré una clave SSH y secretos en GitHub para que GitHub Actions pudiera conectarse al servidor. Finalmente, agregué el job Deploy: cuando TEST y BUILD terminan correctamente, GitHub Actions se conecta a Ubuntu, hace git pull y reconstruye el contenedor. Así, un git push puede llevar una nueva versión hasta el servidor automáticamente.”

Referencia de la práctica

Esta guía se basa en el flujo de la práctica proporcionada: Git para versionado, Docker para contenerización, GitHub Actions para automatización, y una máquina Linux accesible por SSH para la etapa de Deploy.